

**QUANTUM FACE RECOGNITION WITH MULTI-GATE
QUANTUM CONVOLUTIONAL NEURAL NETWORK**

Submitted by:

**Subrata Lodh (2314051),
Yesh Agarwal (2314080)**

SUPERVISOR NAME: Dr. Banani Basu

SUPERVISOR'S SIGNATURE



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**

**NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR, ASSAM
(INDIA)-788010**

Abstract

Face recognition is one of the most widely deployed biometric technologies, and deep convolutional neural networks (CNNs) dominate it, but at a steadily growing computational cost [1, 2]. Quantum machine learning offers an alternative, yet today’s noisy intermediate-scale quantum (NISQ) devices provide only a handful of qubits, so a whole face image cannot be encoded with one qubit per pixel [3, 4]. This project studies and implements the multi-gate quantum convolutional neural network (MG-QCNN) of Zhu *et al.* [4], a hybrid model in which a four-qubit variational circuit acts as a convolution filter sliding over the image. Each 2×2 patch is angle-encoded with R_Y gates, processed by trainable R_Z and R_X rotations (eight parameters) and a CNOT chain, and read out as Pauli-Z expectation values; a fully connected layer classifies the resulting $24 \times 24 \times 4$ feature maps.

Up to mid-semester we implemented the complete pipeline in PennyLane and PyTorch, derived a closed-form expression for the filter output and confirmed it against the circuit (maximum error 7×10^{-16}), and analysed the training cost of the hardware-compatible parameter-shift rule against simulator backpropagation, which gives identical gradients (agreement 2.2×10^{-15}). Evaluated with 1000 shots over ten random splits, the model reaches a mean test accuracy of **94.67%** (max 97.50%, std 1.68) on ORL and **85.56%** (max 91.11%, std 4.79) on Yale. On Yale the training accuracy reaches 100% on only 120 images, and data augmentation did not help, which points to over-fitting. Min-max pixel scaling beat Gaussian scaling on both datasets. Remaining work: ablations, noise studies, hardware readiness and a feature-extraction stage to reduce circuit cost.

1. Introduction

Face recognition identifies or verifies a person from a face image and is used in banking, security and government services. A typical system pre-processes the image, extracts facial features and matches them against known identities; feature-extraction quality largely decides accuracy [1]. Classical methods such as KNN and SVM were replaced by deep learning, in particular CNNs [5], whose shared kernels suit images with correlated neighbouring pixels (DeepFace: 97.35% on LFW [6]), but at ever larger compute budgets [2].

Quantum computing is a candidate for more efficient learning [3, 7], but current NISQ hardware offers only a few tens of reliable qubits, so a practical approach must extract features from small regions with few qubits, like a classical kernel that processes one window at a time. The MG-QCNN of Zhu *et al.* [4] follows this idea: a fixed four-qubit variational circuit is used as a “quanvolutional” filter on every 2×2 patch. The objectives of the project are:

1. To study quantum machine learning: angle encoding, variational circuits and quanvolution.
2. To implement the MG-QCNN with PennyLane and PyTorch and derive its filter function analytically.
3. To evaluate it on the ORL and Yale databases with a statistically sound protocol (ten random splits, finite-shot testing).
4. To analyse the role of gates, entanglement, filter size, shot noise and hardware cost.

2. Literature Review

Classical face recognition. Hand-crafted features (Gabor responses, elastic graph matching) with KNN or SVM gave way to deep CNNs [1], whose computational requirements grow faster than hardware efficiency [2]; this motivates models with far fewer trainable resources. **Variational circuits and quanvolution.** Variational quantum algorithms train a parameterised circuit with a classical optimiser: rotation angles act as weights, the output is an expectation value, and parameters are updated by gradient descent [7]. They are the natural building block for hybrid networks on NISQ devices [3]. Highly random or expressive circuits on many qubits suffer from barren plateaus, where gradients vanish exponentially [8, 9], so small, fixed, structured circuits are preferred. Cong *et al.* introduced a QCNN for many-body physics without a sliding filter, which would need one qubit per pixel for images [10]. Henderson *et al.* proposed the quanvolutional network with random, non-trainable filters [11]; Mattern *et al.* added trainable gates (VQNN) but kept random circuits [12]; Oh *et al.* used a fixed circuit with one measured qubit and several kernels [13]. Other hybrids are HQCCNN [14], HQNN [15] and a QCNN for classical data [16]. In [4] these models use 11 to 22 gates and 2,304 (HQNN) to 23,040 (HQCCNN) quantum parameters on Yale, measure either all or a single qubit, and only VQNN uses a random circuit.

Research gap. The MG-QCNN [4] combines a fixed circuit, trainable R_X/R_Z rotations after R_Y encoding, CNOT entanglement, measurement of all qubits and a sliding 2×2 filter. The authors note that only small datasets were used, simulation was noise-free, a 3×3 kernel is already near the barren-plateau limit, and no hardware experiment was run. Independent implementation, a careful statistical evaluation, an analytical understanding of the filter and an analysis of the training cost and noise are therefore the focus of our work.

3. Methodology

3.1 System overview and data

The system is a hybrid network with one quantum convolution layer followed by one classical fully connected layer (Fig. 1). A 48×48 grayscale image is divided into non-overlapping 2×2 patches (stride 2), giving $M = 24 \times 24 = 576$ patches. The same four-qubit circuit is applied to every patch and returns four real numbers, so the layer outputs a $24 \times 24 \times 4$ tensor, which is flattened to 2304 values and mapped to the 40 ORL subjects (15 for Yale) by a linear layer with softmax.

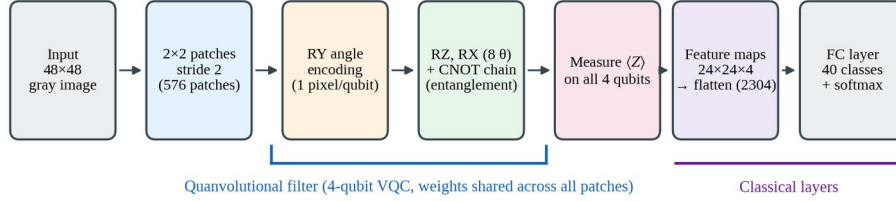


Fig. 1. Overall architecture of the implemented MG-QCNN.

The ORL database [17] has 400 images of 40 subjects (92×112 pixels, 10 per subject, varying expression, lighting and glasses); the Yale database [18] has 165 images of 15 subjects (11 each; 320×243 pixels; centre-/left-/right-light, glasses, happy, sad, sleepy, surprised, wink, normal). Images are converted to grayscale and resized to 48×48 (for Yale a centred square crop followed by a further 85% centre crop). Loaders assert the image counts (Kaggle duplicates and an extra non-ORL subject are removed). Splits are random 70:30 for ORL (280/120) and 120/45 for Yale. Each pixel a is scaled per image by min-max to $[-1, 1]$ and multiplied by π , so it becomes a rotation angle:

$$x_i = \pi \left(\frac{2(a_i - a_{\min})}{a_{\max} - a_{\min}} - 1 \right) \quad (1)$$

Min-max scaling was compared with per-image z-score standardisation clipped to $[-1, 1]$ (the variant described in [4]); on identical splits min-max was higher on all ten seeds for both datasets (Table 2), so it was adopted. A plausible reason is that min-max uses the full angle range for every image, whereas clipping saturates bright and dark pixels at $\pm\pi$; this explanation was not tested separately.

3.2 Quantum convolution layer

Encoder. The four pixels of a patch $\mathbf{x} = (x_0, \dots, x_3)$ are loaded onto four qubits initialised in $|0\rangle$ by R_Y rotations, where for a Pauli operator P the rotation is $R_P(\varphi) = \exp(-i\varphi P/2) = \cos(\varphi/2) I - i \sin(\varphi/2) P$. Each qubit then sits on the Bloch sphere at $(\sin x_i, 0, \cos x_i)$:

$$|\psi_0\rangle = \bigotimes_{i=0}^3 R_Y(x_i) |0\rangle, i=0, \dots, 3 \quad (2)$$

Variational circuit. Each qubit passes through a trainable R_Z and R_X , giving eight parameters $\theta_0 \dots \theta_7$ (θ_{2i} for R_Z , θ_{2i+1} for R_X) shared by all patches, exactly like classical kernel weights. A chain of three CNOT gates entangles neighbouring qubits, so the outputs depend on correlations between pixels. The filter has 15 gates (4 R_Y , 4 R_Z , 4 R_X , 3 CNOT) and depth six (Fig. 2). The circuit is fixed rather than random, which makes results reproducible and helps avoid barren plateaus [8, 9]:

$$|\psi\rangle = C \bigotimes_{i=0}^3 [R_X(\theta_{2i+1}) R_Z(\theta_{2i}) R_Y(x_i)] |0000\rangle, C = \text{CNOT}_{2 \rightarrow 3} \text{CNOT}_{1 \rightarrow 2} \text{CNOT}_{0 \rightarrow 1} \quad (3)$$

Decoder. All four qubits are measured and the Pauli-Z expectation value of each is the feature value (exact in training, estimated from 1000 shots at test time):

$$f_k(x, \theta) = \langle \psi | Z_k | \psi \rangle \quad (4)$$

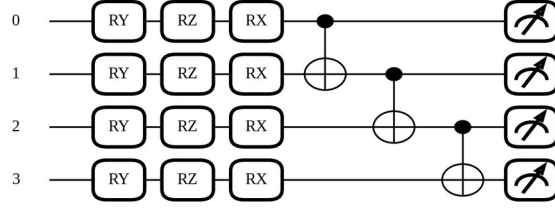


Fig. 2. Four-qubit MG-QCNN filter drawn from the implemented PennyLane circuit.

Closed-form filter output. The pre-CNOT state is a product state. For one qubit, $R_Y(x)$ places the Bloch vector at $(\sin x, 0, \cos x)$; $R_Z(\theta_z)$ rotates it about z and $R_X(\theta_x)$ about x , giving the z -component

$$s_i = \cos \theta_{2i+1} \cos x_i + \sin \theta_{2i+1} \sin \theta_{2i} \sin x_i \quad (5)$$

A CNOT with control c and target t maps $Z_t \rightarrow Z_c Z_t$ and leaves Z_c unchanged. Propagating Z_k back through the chain gives $C^\dagger Z_k C = Z_0 Z_1 \cdots Z_k$, hence

$$f_k = s_0 s_1 \cdots s_k \quad (6)$$

This was checked numerically against the full state-vector circuit on 200 random patches and parameter sets (maximum difference 6.7×10^{-16}). It shows that (i) f_0 depends on pixel 0 only and f_3 on all four pixels, so entanglement is what lets the outputs combine neighbouring pixels (Fig. 3b); without CNOTs each output is just s_k of one pixel; (ii) every output is a bounded sinusoid in each pixel and in each θ (Fig. 3a); and (iii) since each θ enters with frequency one, the parameter-shift rule below is exact. Overall the model is a softmax classifier on 2304 trigonometric pixel features controlled by only eight quantum parameters.

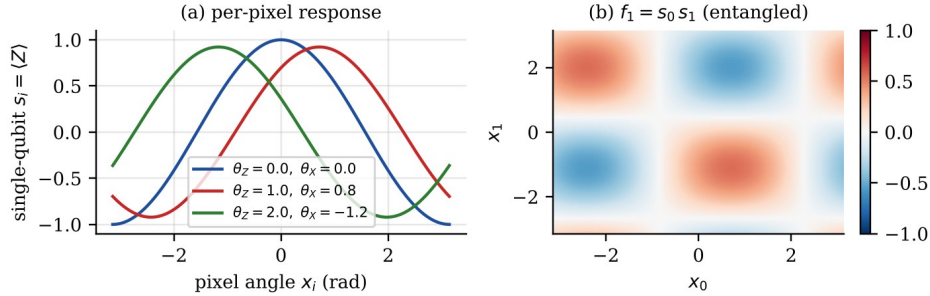


Fig. 3. (a) Single-qubit response $s_i(x_i)$ for three settings of (θ_z, θ_x) . (b) The entangled output $f_1 = s_0 s_1$ as a function of two pixels, using the closed form in Eq. (6).

3.3 Classifier, loss and training

Let $\mathbf{F} \in \mathbb{R}^{2304}$ hold the 576 $\times$ 4 feature values and $\mathbf{z} = \mathbf{WF} + \mathbf{b}$ the logits. The network is trained with softmax cross-entropy over N samples with true classes y_n and K classes (40 or 15):

$$L = -\frac{1}{N} \sum_{n=1}^N \ln \frac{\exp(z_{n, y_n})}{\sum_{c=1}^K \exp(z_{n, c})} \quad (7)$$

Because the filter weights are shared, the gradient for each quantum parameter sums over all patches p :

$$\frac{\partial L}{\partial \theta_m} = \sum_{p=1}^{576} \sum_{k=0}^3 \frac{\partial L}{\partial F_{p, k}} \frac{\partial f_k(x_p)}{\partial \theta_m} \quad (8)$$

The eight quantum parameters and the classical weights are updated together with Adam [19] at learning rate 10^{-3} . For ORL the model has 8 quantum + 2304 $\times$ 40 + 40 = 92,200 classical parameters (92,208 in total); for Yale 8 + 34,575 = 34,583. The classical head therefore holds almost all trainable weights, and the quantum filter determines which features it sees. Table 1 lists the configuration; all runs use ten random splits (seeds 0–9), 1000 test shots, Python, PyTorch 2.11 and PennyLane 0.45.1 on Kaggle.

Table 1. Experimental configuration.

Item	ORL	Yale
Input / filter / stride	48×48 gray / 2×2 / 2	Same, after 85% centre crop
Qubits / quantum params	4 / 8 (shared by all patches)	Same
Feature map / classifier	24×24×4 = 2304 → FC (40)	2304 → FC (15)
Loss / optimiser	Cross-entropy / Adam, lr 10 ⁻³	Cross-entropy / Adam, lr 10 ⁻³
Epochs / batch size	30 / 10	60 / 6
Train / test split	280 / 120 (70:30)	120 / 45

3.4 Gradient computation and cost on hardware

On real hardware the quantum state cannot be read out, so gradients come from the parameter-shift rule [22, 23]: for a gate $\exp(-i\theta P/2)$ with Pauli generator P (R_x, R_z) the exact derivative of an expectation value is

$$\frac{\partial f}{\partial \theta} = \frac{1}{2} [f(\theta + \pi/2) - f(\theta - \pi/2)] \quad (9)$$

This costs two extra runs per parameter. Since all four qubits are measured in one run, one patch needs 1 forward + 2×8 shifted = 17 runs, so one image and one gradient step need

$$N_{\text{runs}} = (1 + 2P)M = 17 \times 576 = 9,792, \quad (10)$$

with $P = 8$ parameters and $M = 576$ patches. Over an ORL training run (280 images × 30 epochs) this is about 8.2×10^7 circuit executions, and about 3.5×10^7 for Yale; at 1000 shots each the ORL run alone would need about 8.2×10^{10} measurements, before any queue time on cloud hardware. Inference needs only the $M = 576$ forward runs per image (Fig. 4a). The cost grows linearly with the number of patches.

A state-vector simulator stores the whole state, so the same function is differentiated by backpropagation in one forward and backward pass with all patches of a mini-batch processed together. Both give the exact derivative of the same noise-free expectation value, so the gradients are mathematically identical; on 20 random patches the eight components agree to 2.2×10^{-15} . Backpropagation is therefore used for training, purely for speed (about 1.3 s per epoch and 40 s per 30-epoch ORL run), and the layer is run with 1000 shots at test time. Each expectation estimated from N_s shots has standard deviation

$$\sigma = \sqrt{\frac{1 - f^2}{N_s}} \leq \frac{1}{\sqrt{N_s}} \approx 0.032 \text{ for } N_s = 1000 \quad (11)$$

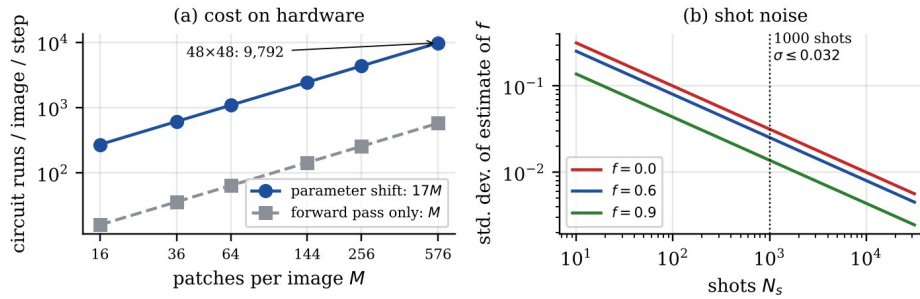


Fig. 4. (a) Circuit runs per image and gradient step against the number of patches M . (b) Standard error of a Pauli-Z estimate against the number of shots.

This is a classical simulation: a real implementation would submit the same circuit to a processor and train with the parameter-shift rule on shot-based estimates, where gate, decoherence and read-out errors also affect gradients and accuracy.

Differences from the reference protocol [4]. Pixel scaling: min–max instead of Gaussian (tested, Section 4.2). Yale optimiser setting: lr 10⁻³ and 60 epochs, selected on a validation split, instead of 10⁻⁴ and 30 epochs. Mini-batch: 10 (ORL) and 6 (Yale) images. Everything else (circuit, 2×2 filter, stride 2, Adam, 1000 shots, 7:3 random splits) follows [4].

4. Work Done Till Mid-Semester and Results

4.1 Implementation and training behaviour

The pipeline was implemented in Kaggle notebooks (one per dataset, same model code): a data loader with count checks; the quantum layer as a PyTorch module with an exact mode (backpropagation) for training and a 1000-shot mode for testing, where all patches of a mini-batch are sent to the circuit in one broadcast call; the full model; and a ten-seed loop

with a new split, fresh initialisation and full training run per seed. The circuit was verified independently: 15 gates as in [4], eight trainable parameters, outputs within $[-1, 1]$, matching backpropagation and parameter-shift gradients, and agreement with Eq. (6).

For ORL seed 0 (Fig. 5) the loss falls from 3.03 in epoch 1 to 0.0031 at epoch 30. Training accuracy reaches 100% in epoch 3, while test accuracy rises from 63.33% (epoch 1) to 91.67% (epoch 2), exceeds 95% from epoch 7 and stays at 95.83% from epoch 14 (115 of 120 images). The 4.17-point train–test gap with 92,200 classical weights on 280 images shows some over-fitting.

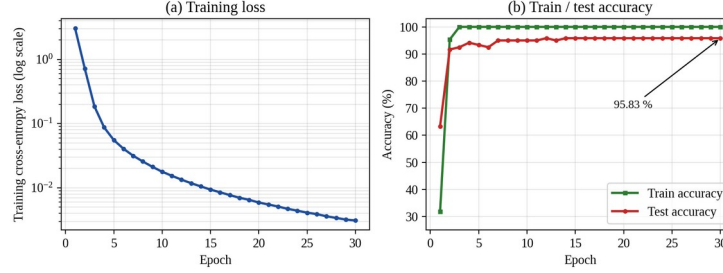


Fig. 5. Training of MG-QCNN on ORL, seed 0 (30 epochs, $\text{lr } 10^{-3}$): (a) loss, log scale; (b) training and test accuracy.

4.2 Ten-seed evaluation

Table 2, Fig. 6 and Fig. 7 give the 1000-shot test accuracy of each split. On ORL the mean is 94.667% (max 97.500%, std 1.675). On Yale the original set-up (full-frame resize, $\text{lr } 10^{-4}$, 30 epochs) gave 78.889% (max 86.667%, std 5.984) and the selected set-up (85% centre crop, $\text{lr } 10^{-3}$, 60 epochs) 85.556% (max 91.111%, std 4.792). The larger Yale spread is expected: one of 45 test images changes accuracy by 2.22 points, against 0.83 points on ORL (population standard deviations). Gaussian normalisation was lower on every seed, by 3.17 points on average for ORL and 8.44 for the original Yale set-up.

Table 2. Test accuracy (%) per seed, 1000 shots. “Gauss.” is z-score normalisation; the Yale “orig.” set-up is full-frame resize, $\text{lr } 10^{-4}$, 30 epochs; “sel.” is the selected set-up.

Seed	ORL min-max	ORL Gauss.	Yale orig. min-max	Yale orig. Gauss.	Yale sel. min-max
0	95.83	94.17	82.22	77.78	91.11
1	91.67	89.17	82.22	71.11	88.89
2	94.17	91.67	68.89	64.44	82.22
3	95.00	90.00	77.78	71.11	82.22
4	94.17	92.50	82.22	71.11	84.44
5	97.50	92.50	86.67	75.56	91.11
6	96.67	90.83	84.44	68.89	84.44
7	94.17	92.50	71.11	64.44	84.44
8	95.00	91.67	82.22	80.00	91.11
9	92.50	90.00	71.11	60.00	75.56
Mean	94.667	91.500	78.889	70.444	85.556
Std.	± 1.675	± 1.434	± 5.984	± 5.967	± 4.792

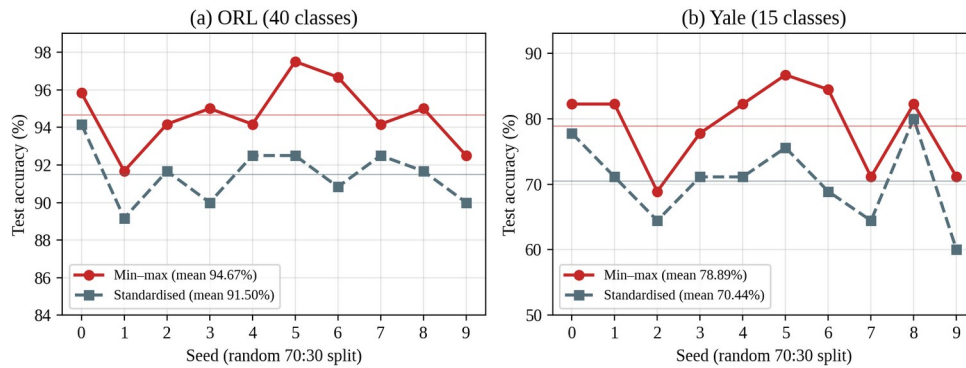


Fig. 6. Test accuracy of the ten seeds for min-max scaling (red) and Gaussian normalisation (grey) on (a) ORL and (b) Yale (original Yale set-up); thin horizontal lines mark the means.

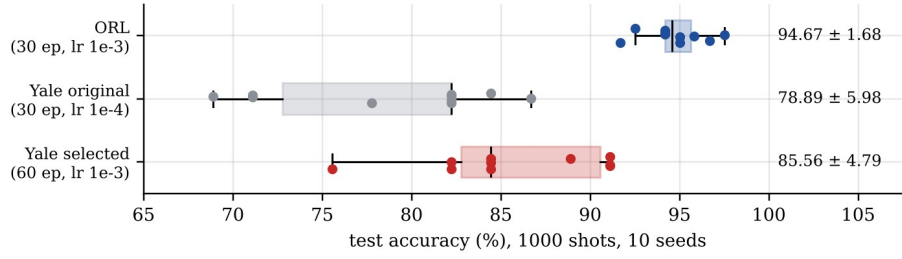


Fig. 7. Distribution of the ten per-seed test accuracies (box = quartiles, dots = seeds; mean $\pm$ std at right).

4.3 Yale tuning and augmentation study

Because the original Yale result was low, pre-processing (full-frame resize, centre square crop, tighter 80% crop), learning rate (10^{-4} or 10^{-3}) and epochs (30 or 60) were varied on a validation split (96 training / 24 validation images from the training part of seeds 0–2; test images unused). Fig. 8a shows that the best setting was the square crop with lr 10^{-3} and 60 epochs (76.4%); the training accuracy reached 100% in every case after 60 epochs. On the ten test splits it gave 84.889% (std 6.577); the slightly tighter 85% crop gave the 85.556% used above, within run-to-run variation.

A further study tested training-time augmentation (random crop, mirror flip, $\pm 10^\circ$ rotation, brightness/contrast jitter) with optional dropout (0.3) and weight decay (10^{-4}), 80 epochs, validation split as before (mean over seeds 0–2, last three epochs). No augmentation scored best, 77.8%; crop+flip gave 71.8%, full augmentation 69.0% and full augmentation with dropout and weight decay 68.1% (Fig. 8b). The gap between clean training accuracy (97–100%) and validation accuracy grew from 22 to 29 points. Augmentation therefore did not reduce the over-fitting; one possible reason is that the large linear head still memorises the training images, but this was not tested.

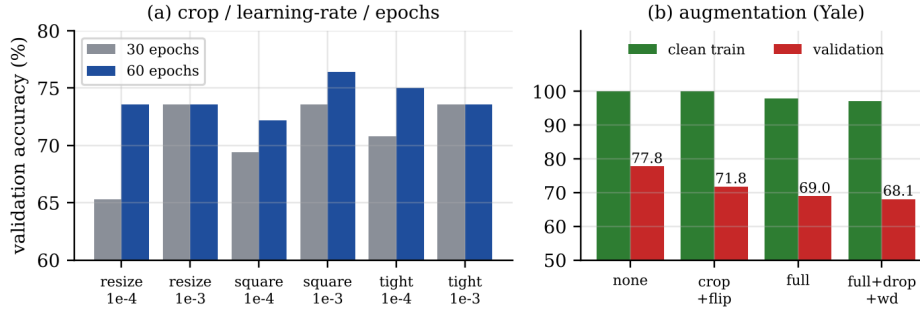


Fig. 8. Yale validation studies. (a) Pre-processing, learning rate and epochs (96/24 split, seeds 0–2). (b) Augmentation: clean training accuracy against validation accuracy.

4.4 Confusion matrices

Fig. 9 shows pooled confusion matrices over ten seeds (1,200 ORL and 450 Yale predictions; Yale class n is subject $n+1$). On ORL the errors are scattered single confusions with no systematically misclassified subject. On Yale the off-diagonal confusion is larger; in this tuning run subjects 7 and 8 are often predicted as subject 14, and subject 5 as subject 13.

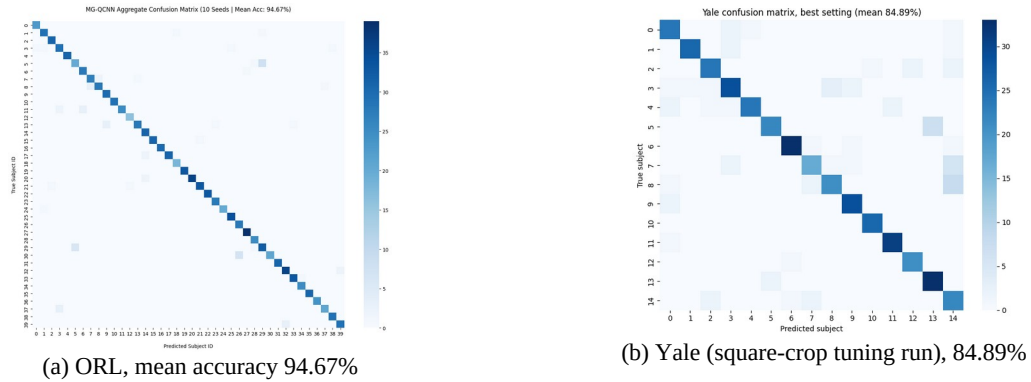


Fig. 9. Aggregate confusion matrices of MG-QCNN over ten seeds.

5. Limitations

The quantum part is a noise-free classical simulation with shot noise only at test time, so the numbers are not hardware results. The model over-fits on Yale (100% training accuracy on 120 images against about 85% test accuracy) and

augmentation did not help. No ablation or noise study has been run yet, and test loss, precision, recall and F1-score are not yet reported. The final epoch is reported on the test set; a validation split was used for the Yale studies only.

6. Future Scope

Training here uses simulator backpropagation, which is fast but is not how a quantum processor is trained. Mathematically the gradients are identical to those of the parameter-shift rule, but in practice a real implementation must use the latter, and with 17 circuit runs per patch (9,792 per image per step, Eq. 10) training becomes very slow, as is also noted in the reference work [4]. A natural extension is to add a feature-extraction stage (for example a small learned or fixed classical front-end) that compresses the image before the quantum layer. Since the cost scales linearly with the number of patches M (Fig. 4a), reducing 576 patches to 144 or 36 would cut the circuit executions by 4 $\times$ or 16 $\times$, and with a pretrained or fixed front-end only the eight quantum parameters would need shifted runs. An ablation should confirm that the quantum layer still contributes.

References

- [1] M. Wang and W. Deng, “Deep face recognition: A survey,” *Neurocomputing*, vol. 429, pp. 215–244, Mar. 2021.
- [2] N. C. Thompson, K. Greenewald, K. Lee, and G. F. Manso, “The computational limits of deep learning,” arXiv:2007.05558, 2020.
- [3] J. Preskill, “Quantum computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, Aug. 2018.
- [4] Y. Zhu, A. Bouridane, M. E. Celebi, D. Konar, P. Angelov, Q. Ni, and R. Jiang, “Quantum face recognition with multigate quantum convolutional neural network,” *IEEE Trans. Artif. Intell.*, vol. 5, no. 12, pp. 6330–6341, Dec. 2024.
- [5] Y. LeCun *et al.*, “Backpropagation applied to handwritten zip code recognition,” *Neural Comput.*, vol. 1, no. 4, pp. 541–551, 1989.
- [6] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, “DeepFace: Closing the gap to human-level performance in face verification,” in *Proc. IEEE CVPR*, 2014, pp. 1701–1708.
- [7] M. Cerezo *et al.*, “Variational quantum algorithms,” *Nat. Rev. Phys.*, vol. 3, no. 9, pp. 625–644, 2021.
- [8] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, “Barren plateaus in quantum neural network training landscapes,” *Nat. Commun.*, vol. 9, Art. no. 4812, 2018.
- [9] Z. Holmes, K. Sharma, M. Cerezo, and P. J. Coles, “Connecting ansatz expressibility to gradient magnitudes and barren plateaus,” *PRX Quantum*, vol. 3, no. 1, Art. no. 010313, 2022.
- [10] I. Cong, S. Choi, and M. D. Lukin, “Quantum convolutional neural networks,” *Nat. Phys.*, vol. 15, no. 12, pp. 1273–1278, 2019.
- [11] M. Henderson, S. Shakya, S. Pradhan, and T. Cook, “Quantum convolutional neural networks: Powering image recognition with quantum circuits,” *Quantum Mach. Intell.*, vol. 2, no. 1, 2020.
- [12] D. Mattern *et al.*, “Variational quantum convolutional neural networks with enhanced image encoding,” arXiv:2106.07327, 2021.
- [13] S. Oh, J. Choi, and J. Kim, “A tutorial on quantum convolutional neural networks (QCNN),” in *Proc. ICTC*, 2020, pp. 236–239.
- [14] S. Y.-C. Chen, T.-C. Wei, C. Zhang, H. Yu, and S. Yoo, “Quantum convolutional neural networks for high energy physics data analysis,” *Phys. Rev. Res.*, vol. 4, no. 1, Art. no. 013231, 2022.
- [15] A. Senokosov, A. Sedykh, A. Saginalieva, and A. Melnikov, “Quantum machine learning for image classification,” arXiv:2304.09224, 2023.
- [16] T. Hur, L. Kim, and D. K. Park, “Quantum convolutional neural network for classical data classification,” *Quantum Mach. Intell.*, vol. 4, no. 1, Art. no. 3, 2022.
- [17] F. S. Samaria and A. C. Harter, “Parameterisation of a stochastic model for human face identification,” in *Proc. IEEE Workshop Appl. Comput. Vis.*, 1994, pp. 138–142.
- [18] A. S. Georgiades, P. N. Belhumeur, and D. J. Kriegman, “From few to many: Generative models for recognition under variable pose and illumination,” in *Proc. IEEE Int. Conf. Autom. Face Gesture Recognit.*, 2000, pp. 277–284.
- [19] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [20] V. Bergholm *et al.*, “PennyLane: Automatic differentiation of hybrid quantum-classical computations,” arXiv:1811.04968, 2018.
- [21] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Proc. NeurIPS*, vol. 32, 2019, pp. 8026–8037.
- [22] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, “Quantum circuit learning,” *Phys. Rev. A*, vol. 98, no. 3, Art. no. 032309, 2018.
- [23] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, “Evaluating analytic gradients on quantum hardware,” *Phys. Rev. A*, vol. 99, no. 3, Art. no. 032331, 2019.